
Chapter Six

Software Quality Assurance and Reliability

Software Reliability

- **IEEE-Std-729-1991:** “**Software reliability** is defined as the probability of **failure-free operation** for a specified *period of time* in a specified *environment*”.
- **ISO9126:** “**Reliability** is the capability of the software product to maintain a specified level of performance when used under specified conditions”.
- **Informal:** Reliability is a measure of how well the software provides the services expected by the customer.
- **Quantification:** Number of failures, severity.

Cont'd

- Reliability is one of the metrics that are used ***to measure quality***.
- It is a user-oriented quality factor relating to system operation.
 - » Intuitively, if the users of a system rarely experience failure, the system is considered to be more reliable than one that fails more often.
- A system without faults is considered to be highly reliable.
 - » Constructing a correct system is a difficult task.
 - » Even an incorrect system may be considered to be reliable if the frequency of failure is “acceptable.”

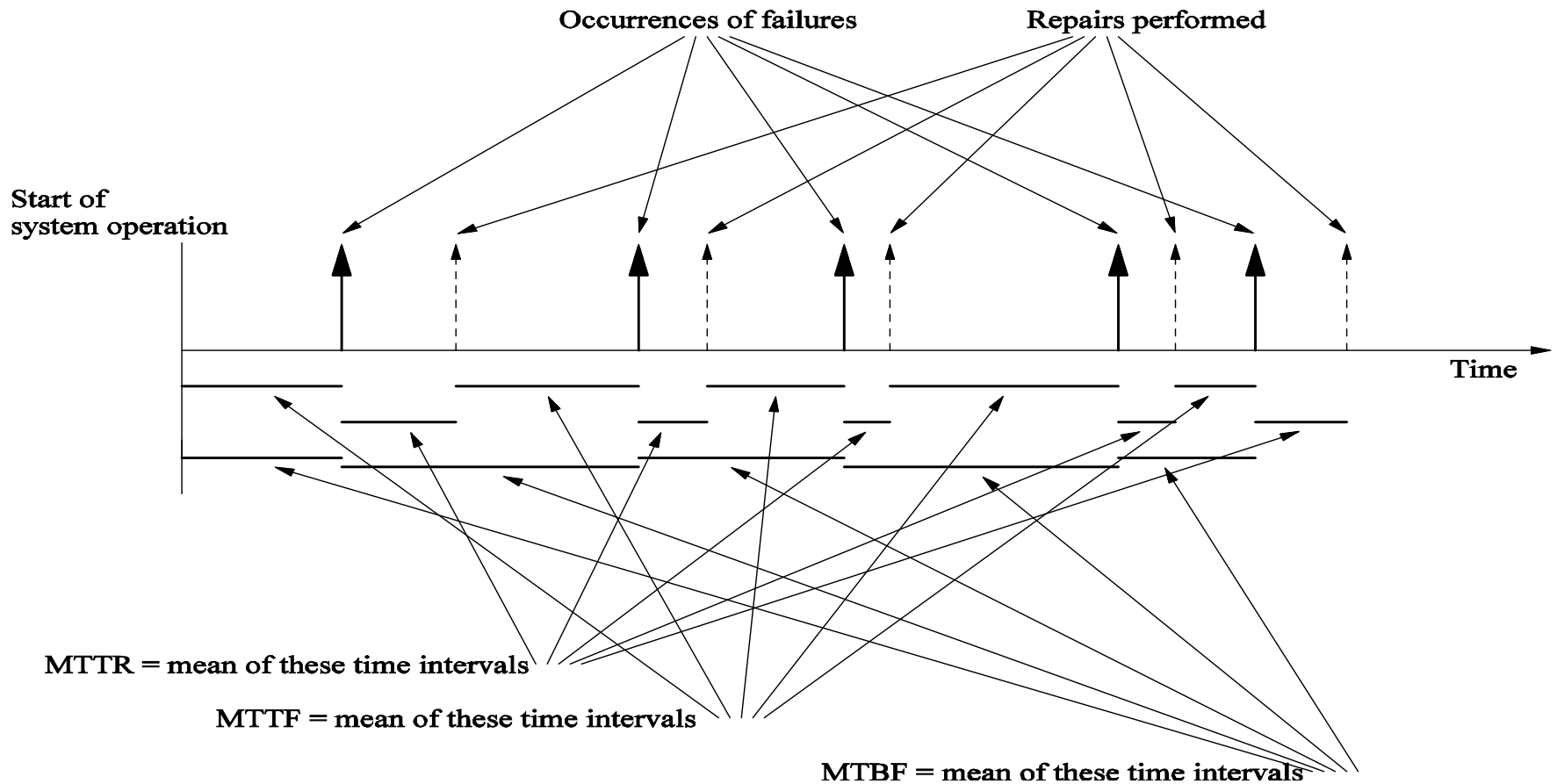
Cont'd

- Key concepts in discussing reliability:
 - » Fault
 - » Failure
 - » Time
 - » Three kinds of time intervals: MTTR, MTTF, MTBF
- Failure
 - » A failure is said to occur if the **observable** outcome of a **program execution** is different from the **expected outcome**.
- Fault
 - » The adjudged ***cause of failure*** is called a fault.
 - » Example: A failure may be caused by a defective block of code.

Cont'd

- Time
 - » Time is a key concept in the formulation of reliability.
 - » If the time gap between two successive failures is **short**, we say that the system is **less reliable**.
 - » Two forms of time are considered.
 - Execution time (τ)
 - Calendar time (t)
- MTTF: Mean Time To Failure
- MTTR: Mean Time To Repair
- MTBF: Mean Time Between Failures (= MTTF + MTTR)

Cont'd



Relationship between MTTR, MTTF, and MTBF

Measuring Software Reliability

- Two ways to measure reliability
 - » Counting failures in periodic intervals
 - Observe the trend of **cumulative failure count** - $\mu(\tau)$.
 - » Failure intensity
 - Observe the trend of **number of failures per unit time** - $\lambda(\tau)$.
- $\mu(\tau)$
 - » This denotes the **total number of failures** observed until execution time τ from the beginning of system execution.
- $\lambda(\tau)$
 - » This denotes the **number of failures observed per unit time** after τ time units of executing the system from the beginning. This is also called the ***failure intensity*** at time τ .
- Relationship between $\lambda(\tau)$ and $\mu(\tau)$
 - » $\lambda(\tau) = d\mu(\tau)/d\tau$

Software Reliability Metrics

- Reliability metrics are ***units of measure*** for system reliability.
- System reliability is measured by ***counting the number of operational failures*** and relating these to demands made on the system at the time of failure.
- A long-term measurement program is required to assess the reliability of critical systems.
- Probability of Failure on Demand (POFOD)
 - » $POFOD = 0.001$ - means 1 out of 1000 service requests result in failure (for one in every 1000 requests the service fails per time unit).
 - » Probability system will fail when a service request is made.
 - » Useful when requests are made on an intermittent or infrequent basis.
 - » Appropriate for protection systems service requests may be rare and consequences can be serious if service is not delivered.
 - » Relevant for many ***safety-critical systems*** with exception handlers

Cont'd

- Rate of Fault Occurrence (ROCOF)
 - » $\text{ROCOF} = 0.02$ - two failures for each 100 operational time units of operation.
 - » Reflects rate of failure in the system.
 - » Useful when system has to process a large number of similar requests that are relatively frequent.
 - » Relevant for operating systems and transaction processing systems.
- Mean Time to Failure (MTTF)
 - » Average time between observed failures.
 - » Measures time between observable system failures.
 - » For stable systems $\text{MTTF} = 1/\text{ROCOF}$
 - » Relevant for systems when individual transactions take lots of processing time (e.g. CAD or WP systems)

Cont'd

- Availability
 - » Measures the fraction of time system is really available for use.
 - » Takes repair and restart times into account.
 - » Relevant for non-stop continuously running systems (e.g. traffic signal)
 - » $MTBF = \text{Mean Time Between Failure}$
 - » $MTTR = \text{Mean Time to Repair}$
 - » $\text{Availability} = MTBF / (MTBF + MTTR)$
- Reliability = $MTBF / (1 + MTBF)$
- Time Units
 - » Raw Execution Time - non-stop system.
 - » Calendar Time - if the system has regular usage patterns.
 - » Number of Transactions - demand type transaction systems.

Failure Classification

- **Transient** - only occurs with certain inputs
- **Permanent** - occurs on all inputs
- **Recoverable** - system can recover without operator help.
- **Unrecoverable** - operator has to help.
- **Non-corrupting** - failure does not corrupt system state or data.
- **Corrupting** - system state or data are altered.

Failure Class	Example	Metric
Permanent Non-corrupting	ATM fails to operate with any card, must restart to correct	ROCOF = .0001 Time unit = days
Transient Non-corrupting	Magnetic stripe can't be read on undamaged card	POFOD = .0001 Time unit = transactions

Software Reliability Models

- Software developers are struggling to deliver **reliable** software with **acceptable level of quality**, within given budget and schedule.
- Therefore, researchers are focusing on the identification of the **number of fault presents** in the software or identification of program modules that are most likely to contain faults.
- Detecting software faults **early during development** will definitely **improve the reliability and quality** in cost-effective way.
- A number of analytical models are developed that are based mainly on the **failure history of software** and can be classified according to the **nature of the failure process**.

Cont'd

- **Software reliability models** usually refer to estimating the number of remaining errors in a partially debugged software.
- Software reliability models can be broadly categorized as suggested by Pham (2006):
 - » **Deterministic** - used to study the number of distinct operators and operands as well as the machine instructions in the program.
 - Ex. Halstead's software metric and McCabe's cyclomatic complexity metric.
 - » **Probabilistic** - describes the failure occurrence and/or fault removal phenomenon of the testing process as probabilistic events with respect to time and/or testing effort.
 - Failure rate model (times between failure models),
 - Failure or fault count model (NHPP models).
 - Error or fault-seeding model.
 - Reliability growth model, etc.

Failure Rate Models

(Time between Failure)

- It assumes that the time between $(i-1)^{\text{th}}$ and the i^{th} failures follows a distribution whose parameters depend on the number of faults remaining in the program during this interval.
- Estimates of the parameters are obtained from the observed values of time between failures or failure count in intervals.
- The estimates of software reliability, mean time to next failure, etc., are then obtained from the fitted model.
- List of some failure rate models (Goel 1985):
 - » Jelinski and Moranda Model.
 - » Schick and Wolverton Model.
 - » Goel and Okumoto Imperfect Debugging Model, etc.

Failure or Fault Count Models

- The interest of this class of models is in the ***number of faults or failures*** in specified time intervals rather than in times between failures.
- The failure counts are assumed to follow a known stochastic process with a time-dependent discrete or continuous failure rate.
- Parameters of the failure rate can be estimated from the observed values of failure counts or from failure times.
- Estimates of software reliability, mean time to next failure, etc., can again be obtained from the relevant equations.

Cont'd

- The key models in this class are Goel (1985):
 - » Shooman exponential model.
 - » Musa execution time model.
 - » Goel–Okumoto non-homogeneous Poisson process model (G–O–NHPP).
 - » S-shape growth model.
 - » Discrete reliability growth model.
 - » Musa–Okumoto logarithmic Poisson execution model.
 - » Generalized NHPP, etc.

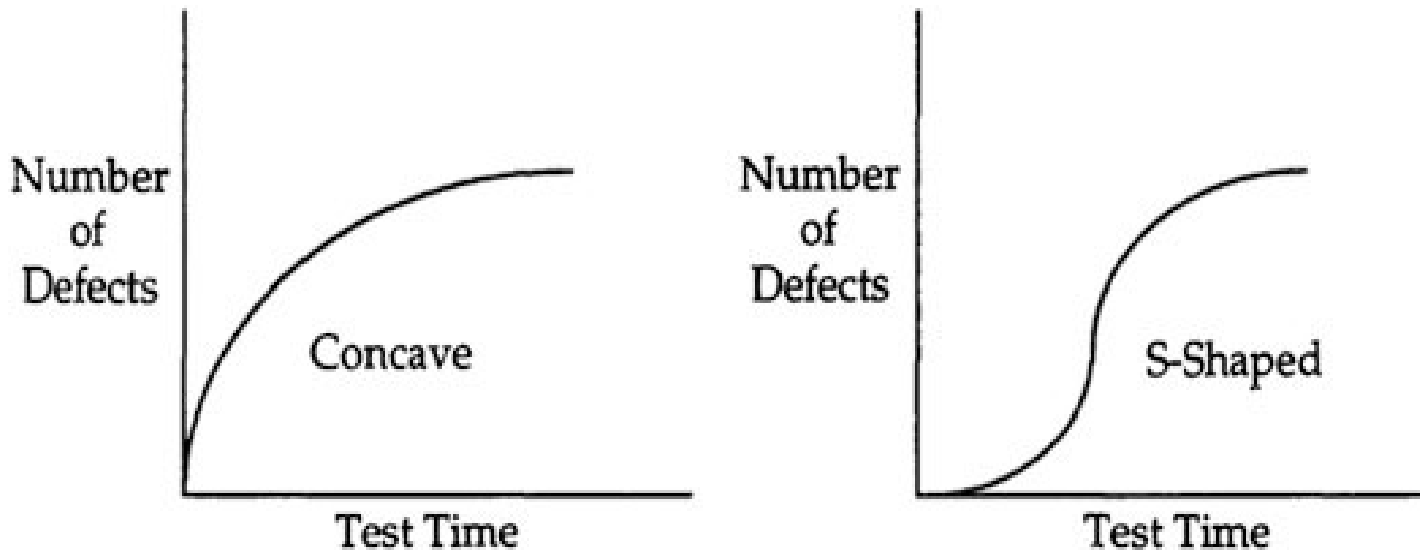
Error or Fault-Seeding Models

- The basic approach in this class of models is to “**seed**” known number of faults in the program which is assumed to have an unknown number of indigenous faults.
- The program is tested and observed for numbers of seeded and indigenous faults.
- From these, an estimate of the indigenous fault content of the program is obtained and used to assess software reliability and other relevant measures.
- Examples:
 - » Mills Hyper geometric model

Reliability Growth Models

- **Software reliability growth** is defined as the phenomenon that the number of software errors remaining in the system decreases with the progress of testing.
- It is applicable during the **testing phase** and quantifies the software reliability in terms of estimated number of software error remaining in software or estimated time intervals between software failures.
- Software reliability growth models have been grouped into two classes of models: **concave and S-shaped**.
- They have the **same asymptotic behavior**, that is, the defect detection rate decreases as the number of defects detected (and repaired) increases, and the total number of defects detected approach a finite value asymptotically.

Cont'd



Concave and S-shaped model

Software reliability growth models (Wood 1996)

Model name	Model type	$\mu(t)$	Comments
Goel–Okumoto (G–O)	Concave	$a(1 - e^{-bt})$ $a \geq 0, b > 0$	Also called Musa model or exponential model
G–O S-shaped	S-shaped	$a(1 - (1 + bt)e^{-bt})$ $a \geq 0, b > 0$	Modification of G–O model to make it S-shaped
Hossain Dahiya	Concave	$a(1 - e^{-bt}) / (1 + ce^{-bt})$ $a \geq 0, b > 0, c > 0$	Becomes same as G–O as c approaches to 0
Yamada exponential	Concave	$a(1 - e^{-r\alpha(1 - e^{-\beta t})})$ $a \geq 0, r\alpha > 0, \beta > 0$	Attempts to account for testing effort
Yamada Raleigh	S-shaped	$a(1 - e^{-r\alpha(1 - e^{(-\beta^2/2)t})})$ $a \geq 0, r\alpha > 0, \beta > 0$	Attempts to account for testing effort
Weibull	Concave	$a(1 - e^{-bt^c})$ $a \geq 0, b > 0, c > 0$	Same as G–O for $c = 1$
Kapur et al.	Concave	$a(1 - e^{-bpt})$	Based on imperfect debugging

Software Quality Assurance

- What is **quality**?
 - » **IEEE Glossary:** Degree to which a system, component, or process meets (1) specified requirements, and (2) customer or user needs or expectations.
 - » **ISO:** the totality of features and characteristics of a product or service that bear on its ability to satisfy specified or implied needs.
 - » **Traditional definition:** fitness of purpose, a quality product does exactly what the users want it to do.
- Quality Criteria include:
 - » Correctness,
 - » efficiency,
 - » flexibility,
 - » integrity,
 - » interoperability,
 - » maintainability,
 - » portability, reliability, reusability, testability and usability

Cont'd

- Software quality assurance (SQA)
 - » Consists of a means of **monitoring** the software engineering processes and methods used **to ensure quality**.
 - » It does this by means of audits of the quality management system under which the software system is created.
 - » These audits are backed by one or more standards, usually ISO 9000 or CMMI.
- It is practically impossible to iron out every single bug before releasing it both from a difficulty point of view and due to time constraints.

Quality Assurance (QA)

- Measures the ***quality of processes*** used to create a quality product.
 - » A set of activities designed to ensure that the development and/or maintenance process is adequate to ensure a system will meet its objectives.
 - » QA activities ensure that the process is defined and appropriate.
 - » Methodology and standards development are examples of QA activities.
 - » A QA review would focus on the process elements of a project

Quality Control (QC)

- Measures the ***quality of a product***.
 - » A set of activities (inspections, reviews, and test) designed to evaluate a developed work product.
 - » QC activities focus on finding defects in specific deliverables.
- Two types of quality control:
 - » **Quality design** - the characteristics that designers specify for an item.
 - includes: requirements, specifications, and the design of the system.
 - » **Quality of conformance** - the degree to which the design specification are followed.
 - It focuses on implementation based on the design.
- Generally, QC compare the work products with the specified and measurable standards.

Cont'd

- Testing - Quality control
 - » The process of executing a system with the intent of finding defects.
 - » includes test planning prior to the execution of the test cases.
 - » Testing is one example of a QC activity, but there are others such as inspections

Quality Assurance Vs. Quality Control

- Both QA and QC activities are generally required for successful software development.
- Software quality control is a control of products.
- Software quality assurance is a control of processes.
 - » Related to the practice of quality assurance in product manufacturing
- Software vs. manufactured product
 - » Manufactured product is physical and can be seen, Software product is not visible.
 - » Manufactured product rolls off the assembly line, it is essentially a complete, finished product, Software is never finished.
- The processes and methods to manage, monitor, and measure software ongoing quality are as fluid and sometimes elusive as are the defects that they are meant to keep in check.

Software Quality Assurance Jobs

- Includes:
 - » Reviewing requirements documents
 - » Software testing
- It also encompasses the entire software development process
 - » Software design
 - » Coding
 - » Source code control
 - » Code reviews
 - » Change management
 - » Configuration management, Release management.

Software Quality Assurance Methodology

- Process and Product Quality Assurance (PPQA) audits:
 - » the activity of ensuring that the process and work product conform to the agreed upon process.

Quality Control Activities

- Peer Reviews:
 - » Peer reviews of a project's work products are the most efficient defect removal (quality control) activity.
- Validation testing
 - » the act of entering data that the tester knows to be erroneous into an application.
 - Example, typing "Hello" into an edit box that is expecting to receive a numeric entry.
- Data comparison
 - » Comparing the output of an application with specific parameters to a previously created set of data with the same parameters that is known to be accurate.

Cont'd

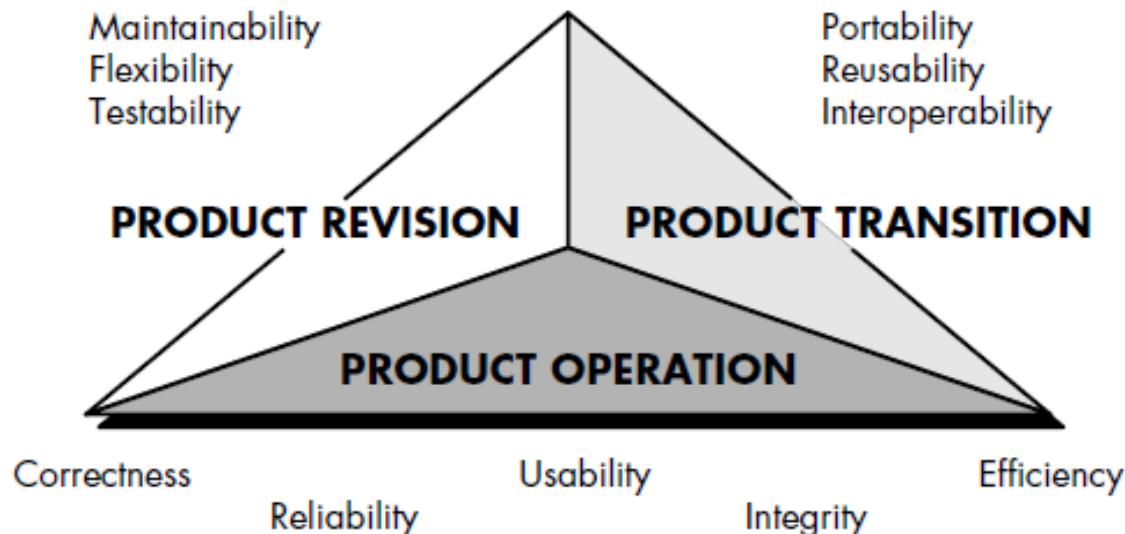
- Stress testing:
 - » A stress test is when the software is used as heavily as possible for a period of time to see whether it copes with high levels of load.
 - » Often used for server software that will have multiple users connected to it simultaneously.
 - » Also known as Destruction testing.
- Usability testing
 - » Sometimes getting users who are unfamiliar with the software to try it for a while and offer feedback to the developers about what they found difficult to do is the best way of making improvements to a user interface

Advantages of Quality Assurance

- Improved customer satisfaction
- Reduced cost of maintenance
- Eliminate errors when they are still inexpensive to correct
- Improves the quality of the software
- Improving the process of creating software

McCall's Quality Factors

- McCall, Richards, and Walters [MCC77] propose a useful categorization of factors that affect software quality.
- These software quality factors focus on three important aspects of a software product:
 - » its operational characteristics,
 - » its ability to undergo change, and
 - » its adaptability to new environments.



Cont'd

- **Correctness** - The extent to which a program satisfies its specification and fulfills the customer's mission objectives.
- **Reliability** - The extent to which a program can be expected to perform its intended function with required precision.
- **Efficiency** - The amount of computing resources and code required by a program to perform its function.
- **Integrity** - Extent to which access to software or data by unauthorized persons can be controlled.
- **Usability** - Effort required to learn, operate, prepare input, and interpret output of a program.

Cont'd

- **Maintainability** - Effort required to locate and fix an error in a program. [This is a very limited definition.]
- **Flexibility** - Effort required to modify an operational program.
- **Testability** - Effort required to test a program to ensure that it performs its intended function.
- **Portability** - Effort required to transfer the program from one hardware and/or software system environment to another.
- **Reusability** - Extent to which a program [or parts of a program] can be reused in other applications—related to the packaging and scope of the functions that the program performs.
- **Interoperability** - Effort required to couple one system to another.

SQA Group

- Who involves quality assurance activities?
 - » Software engineers, project managers, customers, sale people, SQA group
- Engineers involved the quality assurance work:
 - » apply technical methods and measures
 - » conduct formal technical review
 - » perform well-planned software testing
- The SQA group's role
 - » serves as the customer's in-house representative
 - » assist the software engineering team in achieving high-quality

Cont'd

- The SQA group's responsibility:
 - » quality assurance planning oversight, record keeping, analysis and reporting
- The SQA group's tasks:
 - » Prepare a SQA plan for a project.
 - » Participate in the development of the project's software process description
 - » Review engineering activities to verify compliance with the defined process
 - » Audits designated software work products to verify compliance the defined process
 - » Ensure the deviations in software work and products according to a documented procedure
 - » Records any noncompliance and reports to senior management

SQA Plan

- The SQA plan provides a **road map** for instituting software quality assurance.
- Below presents an outline for SQA plans by IEEE [IEEE94].
- Basic items:
 - » purpose of plan and its scope
 - » management
 - organization structure, SQA tasks, their placement in the process
 - roles and responsibilities related to product quality
 - » Documentation
 - project documents, models, technical documents, user documents.
 - » standards, practices, and conventions
 - » reviews and audits
 - » test
 - test plan and procedure
 - » problem reporting, and correction actions
 - » Tools, code control, media control, supplier control,
 - » records collection, maintenance, and retention
 - » Training and risk management

SEI Capability Maturity Model

- Developed by Software Engineering Institute (SEI) of the Carnegie Mellon University, USA:
 - » to assist the U.S. Department of Defense (DoD) in software acquisition.
 - » The rationale was to include:
 - likely contractor performance as a factor in contract awards.
- Major DoD contractors began CMM-based process improvement initiatives:
 - » as they vied for DoD contracts.
- SEI CMM helped organizations:
 - » Improve quality of software they developed
 - » Realize adoption of SEI CMM model had significant business benefits.
- Other organizations adopted CMM.

Cont'd

- In simple words,
 - » CMM is a model for apprising the software process maturity of a contractor into different levels.
 - » Can be used to predict the most likely outcome to be expected from the next project that the organization undertakes.
- Can be used in two ways:
 - » Capability evaluation
 - » Software process assessment.

Cont'd

- Capability Evaluation

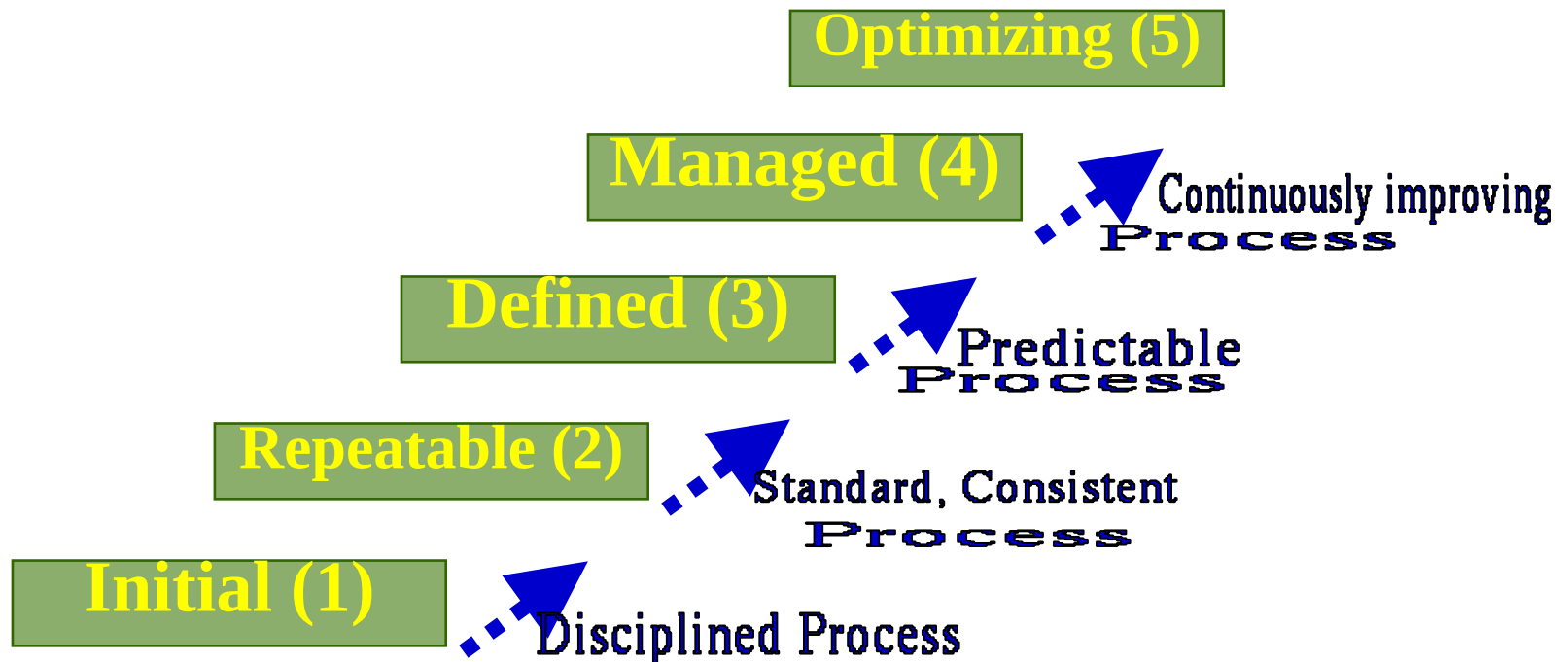
- » Provides a way to **assess** the software process capability of an organization
 - Helps in selecting a contractor
 - Indicates the likely contractor performance

- Software Process Assessment

- » Used by an organization to assess its current process:
 - Suggests ways to improve the process capability.
 - This type of assessment is for purely internal use.

Cont'd

- The SEI CMM classifies software development industries into:
 - ≈ Five maturity levels.
 - ≈ Stages are ordered so that improvements at one stage provide foundations for the next
 - ≈ Based on the pioneering work of Philip Crosby



Level 1: (Initial)

- Organization operates without any formalized process or project plans.
- An organization at this level is characterized by **ad hoc** and often **chaotic** activities.
- Software production processes are not defined,
 - » different engineers follow their own process
 - » development efforts become chaotic.
 - » The success of projects depend on individual efforts and heroics.

Level 2: (Repeatable)

- Basic project management practices
 - » tracking cost, schedule, and functionality are followed.
- Size and cost estimation techniques
 - » function point analysis, COCOMO, etc. used.
- Production process is ad hoc
 - » not formally defined
 - » also not documented.
- Process used for different projects might vary between projects:
 - » earlier success on projects with similar applications can be repeated.
 - » Opportunity to repeat process exist when a company produces a family of products.

Level 3: (Defined)

- Management and development activities:
 - » defined and documented.
 - » Common organization-wide understanding of activities, roles, and responsibilities.
- The process though defined,
 - » process and product qualities are not measured.
- ISO 9001 aims at achieving this level.

Level 4: (Managed)

- Quantitative quality goals for products are set.
- Software process and product quality are measured:
 - » The measured values are used to control the product quality.
 - » Results of measurement used to evaluate project performance
 - rather than improve process.
- Organization sets quantitative quality goals
- World-wide about 100 organizations assessed at this level.

Level 5: (Optimizing)

- Statistics collected from process and product measurements are analysed:
 - » continuous process improvement based on the measurements.
 - Known types of defects are prevented from recurring by tuning the process
 - lessons learned from specific projects incorporated into the process
- Identify best software engineering practices and innovations:
 - » tools, methods, or process are identified
 - » transferred throughout the organization
- World-wide about 50 organizations have been assessed at this level.

Key Process Areas

- Each level is associated with a key process area (KPA) identifies
 - » where an organization at the previous level must focus to reach this level
- Level 2 KPAs
 - » Software project planning
 - Size, cost, schedule.
 - project monitoring
 - » Configuration management
 - » Subcontract management
- Level 3 KPAs
 - » Process definition and documentation
 - » Reviews
 - » Training program

Cont'd

- Level 4 KPAs
 - » Quantitative measurements
 - » Process management
- Level 5 KPAs
 - » Defect prevention
 - » Technology change management
 - » Process change management

ISO 9000

- ISO (international Standards Organization):
 - » a consortium of 63 countries established to formulate and foster standardization.
- ISO published its 9000 series of standards in 1987.
- ISO 9000 certification:
 - » serves as a reference for contract between independent parties.
- The ISO 9000 standard:
 - » specifies guidelines for maintaining a quality system.
- ISO 9000 specifies:
 - » guidelines for repeatable and high quality product development.
 - » Also addresses organizational aspects
 - responsibilities, reporting, procedures, processes, and resources for implementing quality management.

Cont'd

- A set of guidelines for the production process.
 - » not directly concerned about the product it self.
 - » a series of three standards:
 - ISO 9001, ISO 9002, and ISO 9003.
- Based on the premise:
 - » if a proper process is followed for production:
 - good quality products are bound to follow.
- **ISO 9001** - Applies to:
 - » organizations engaged in design, development, production, and servicing of goods.
 - » applicable to most software development organizations.

Cont'd

- **ISO 9002** - applies to:
 - » organizations who do not design products:
 - but are only involved in production.
- **Examples of this category of industries:**
 - » steel or car manufacturing industries
 - » buy the product and plant designs from external sources:
 - only manufacture products.
 - » not applicable to software development organizations.
- **ISO 9003** - applies to:
 - » organizations involved only in installation and testing of the products.

ISO 9000 for Software Industry

- ISO 9000 is a generic standard:
 - » applicable to many industries,
 - starting from a steel manufacturing industry to a service rendering company.
- Many clauses of ISO 9000 documents:
 - » use generic terminologies
 - » very difficult to interpret them in the context of software organizations.

Software vs. Other Industries

- Very difficult to interpret many clauses for software industry:
 - » software development is radically different from development of other products.
- Software is intangible
 - » therefore difficult to control.
 - It is difficult to control anything that we cannot see and feel.
 - » In contrast, in a car manufacturing unit:
 - we can see a product being developed through stages such as fitting engine, fitting doors, etc.
 - one can accurately tell about the status of the product at any time.
 - » Software project management is an altogether different ball game.

Cont'd

- During software development:
 - » the only raw material consumed is data.
- For any other product development:
 - » Lot of raw materials consumed
 - e.g. Steel industry consumes large volumes of iron ore, coal, limestone, etc.
- ISO 9000 standards have many clauses corresponding to raw material control .
 - » not relevant to software organizations.
- Radical differences exist between software and other product development,
 - » difficult to interpret various clauses of the original ISO standard in the context of software industry.

ISO 9000 Part-3

- ISO released a separate document called ISO 9000 part-3 in 1991
 - » to help interpret the ISO standard for software industry.
- At present,
 - » official guidance is inadequate

Why Get ISO 9000 Certification?

- Several benefits:
 - » Confidence of customers in an organization increases
 - if organization qualified for ISO 9001 certification.
 - This is especially true in the international market.
- Many international software development contracts insist:
 - » development organization to have ISO 9000 certification.
- Requires:
 - » a well-documented software production process to be in place.
 - » contributes to repeatable and higher quality software.
- Makes development process:
 - » focussed, efficient, and cost-effective

Cont'd

- Points out the weakness of an organizations:
 - » recommends remedial action.
- Sets the basic framework:
 - » for development of an optimal process and TQM (Total Quality Management).
 - **TQM** – Advocates continuous process improvements through process measurements.
 - optimizes them through redesign.